DD: BIL-05 — Wallet & Topup

Developer implementation guide: `DD_BIL-05-Wallet_and_Topup-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_BIL-05-Wallet_and_Topup-v1.0.md`.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue **Orchestration**: Synchronous service classes per `SUB-WF-FRAMEWORK-01` (Java/Spring OR Laravel/PHP per-module choice). Topup events arrive from external payment gateways; cycle-debits arrive from BIL-03; recovery debits arrive from BIL-04.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-05-Wallet_and_Topup-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `DELETE /api/wallets/{subscriptionId}/aliases/{aliasId}`
- `GET /api/customers/me/wallets`
- `GET /api/gateway-reconciliations?date=YYYY-MM-DD`
- `GET /api/pending-credits?status=HELD&limit=50`
- `GET /api/wallets/{subscriptionId}`
- `GET /api/wallets/{subscriptionId}/aliases`
- `GET /api/wallets/{subscriptionId}/balance`
- `GET /api/wallets/{subscriptionId}/topup-channels`
- `GET /api/wallets/{subscriptionId}/transactions?limit=50`

- `GET /api/wallets?operator={code}&status={status}&limit=50`
- `POST /api/gateway-reconciliations/{id}/resolve`
- `POST /api/gateway/bank/callback`
- `POST /api/gateway/card/callback`
- `POST /api/gateway/mpesa/callback`
- `POST /api/gateway/mtn-momo/callback`
- `POST /api/gateway/retail-agent/callback`
- `POST /api/gateway/vodacom-mpesa/callback`
- `POST /api/pending-credits/{id}/discard`
- `POST /api/pending-credits/{id}/refund-to-gateway`
- `POST /api/pending-credits/{id}/route-to-wallet`
- `POST /api/wallets`
- `POST /api/wallets/{id}/close`
- `POST /api/wallets/{id}/credit`
- `POST /api/wallets/{id}/credit-adjustment`
- `POST /api/wallets/{id}/debit`
- `POST /api/wallets/{id}/debit-adjustment`
- `POST /api/wallets/{id}/suspend`
- `POST /api/wallets/{id}/unsuspend`
- `POST /api/wallets/{subscriptionId}/aliases`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `advance-cycle-on-recovery`
- `billing-wallet`
- `cycle-close`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue

Orchestration: Synchronous service classes per `SUB-WF-FRAMEWORK-01` (Java/Spring OR Laravel/PHP per-module choice). Topup events arrive from external payment gateways; cycle-debits arrive from BIL-03; recovery debits arrive from BIL-04.

A — Target

1. What this process is about

BIL-05 is the **wallet & topup engine** — the module that owns the customer wallet entity for PREPAID Subscriptions. It tracks wallet balance, records all credits (topups, refunds, bonuses, corrections) and debits (cycle charges, recovery debits), exposes balance query and debit/credit operations to other billing modules, and emits Kafka events on every balance-changing transaction for downstream consumers (BIL-04 dunning recovery, notification module, analytics).

For **PREPAID Wananchi customers** (the majority of Wananchi's subscriber base): each Subscription has exactly one wallet. The customer tops up the wallet through external channels (mobile money, bank transfer, retail agent, online card) — those topups arrive at BIL-05 via gateway-specific adapters that translate provider events into uniform `WalletToppedUp` credits.

At each cycle boundary, BIL-03 queries the wallet balance and (if sufficient) calls BIL-05 to debit the cycle charge. If a customer falls into dunning for wallet-insufficient, any subsequent topup triggers BIL-04's auto-recovery path which calls BIL-05 to debit the cycle charge and restore service.

BIL-05 does NOT do invoicing, charging logic, cycle math, or dunning. It is a **balance ledger + topup gateway integration layer**, with strict immutability on transaction records and idempotency on all balance-changing operations.

Out of scope:

- **Cycle charge computation** — owned by BIL-01 / BIL-03. BIL-05 receives a "debit this amount" request and does not validate the amount against the subscription's package.
- **Invoice generation** — owned by BIL-02. BIL-05 does not generate invoices for topups (topup receipts are a separate notification, not an invoice).
- **Dunning state machine** — owned by BIL-04. BIL-05 emits `WalletToppedUp` events that BIL-04 listens to; the dunning level transitions are BIL-04's concern.
- **Cycle close detection** — owned by BIL-03. BIL-05 does not detect when a cycle is due; it receives a debit call when BIL-03 decides the cycle is closing.
- **Refund decision logic** — owned by various initiating modules (TERMINATE workflow for early-termination refunds, BIL-02 for invoice-credit refunds). BIL-05 receives a refund credit request and applies it; it does not decide whether a refund is owed.
- **Payment gateway technical integration details** — owned by infra / shared payment-gateway library. BIL-05 has gateway adapters but the underlying API integration with M-Pesa / MTN Mobile Money / bank rails / card processor is shared infrastructure used by other modules too.
- **Yas Senegal pre-payment invoices** — owned by BIL-02 (which generates them) and BIL-01 (which records payments against them). Yas SN does NOT have wallets in V3 — wallets exist only for PREPAID Wananchi customers. Yas SN PREPAYMENT customers pay invoices directly through the same external channels (mobile money, etc.) but the payment is recorded against the cycle invoice in BIL-01, not in a wallet ledger.
- **Mid-cycle pay-as-you-go purchases** (extra data bundles, voice top-ups, pay-per-view) — out of V3 scope. If a PREPAID customer wants to buy an extra service mid-cycle, that's a V3.2 feature. V3 wallets are debited only for cycle charges.
- **Multi-currency wallet** — V3 wallets are single-currency, matching the Subscription's currency (KES for Wananchi KE, UGX for Wananchi UG, TZS for Wananchi TZ). A customer with Subscriptions in multiple tenants has multiple wallets, one per Subscription.

2. Business rules

Rules grouped by scope: `W` for wallet entity, `T` for transactions, `D` for debits, `C` for credits / topups, `G` for gateway integration, `S` for state interactions.

Wallet entity rules

ID	Rule	Triggered on
R-BIL-05-W-1	A wallet exists for each PREPAID Subscription. Subscriptions with `billing_mode = POSTPAID` or `billing_mode = PREPAYMENT` do NOT have wallets. The wallet is created automatically when SUB-WF-ACTIVATE-01 activates a PREPAID Subscription and is closed automatically when SUB-WF-TERMINATE-01 terminates the Subscription. There is exactly one wallet per Subscription, keyed by `subscription_id`.	Subscription lifecycle
R-BIL-05-W-2	Each wallet has a single currency, matching the Subscription's currency at activation. The currency is immutable for the wallet's lifetime — if a Subscription needed to change currency, that would require terminate-and-recreate. In V3 the supported currencies are KES (Wananchi KE), UGX (Wananchi UG), TZS (Wananchi TZ). XOF (Yas SN) is reserved for V3.2 if Yas SN adds PREPAID alongside its current PREPAYMENT.	Wallet creation

ID	Rule	Triggered on
R-BIL-05-W-3	A wallet has a `status`: `ACTIVE` (normal operation, accepts credits and debits), `SUSPENDED` (operator-frozen — accepts no credits or debits, used during fraud investigation or regulatory hold), `CLOSED` (Subscription terminated — no further operations possible, balance was settled at termination). Status transitions are role-gated (`WALLET_ADMIN` for ACTIVE→SUSPENDED→ACTIVE; `BILLING_INTERNAL` for ACTIVE→CLOSED via the termination workflow).	Wallet state
R-BIL-05-W-4	A wallet has a `balance` (NUMERIC) computed from the sum of all credit transactions minus the sum of all debit transactions. The balance is materialized on the wallet row for fast read access, recalculated atomically with each transaction. Balance MAY NOT go negative — debit operations that would result in a negative balance are rejected with `INSUFFICIENT_BALANCE` (callers must check balance first or accept the rejection).	Balance semantics
R-BIL-05-W-5	The wallet record carries: `subscription_id`, `customer_id`, `operator_code`, `currency`, `status`, `balance`, `created_at`, `closed_at`, plus a denormalized `total_credits_lifetime` and `total_debits_lifetime` for reporting. The latter two are also recomputed atomically per transaction.	Wallet record

Transaction rules

ID	Rule	Triggered on
R-BIL-05-T-1	Every balance-changing operation creates a row in `wallet_transaction`. Transactions are immutable — once posted, they cannot be edited or deleted. Corrections happen as new transactions (reversals or adjustments), never as edits.	All operations
R-BIL-05-T-2	Every transaction has a `type` (`CREDIT_TOPUP_MOBILEMONEY`, `CREDIT_TOPUP_BANK`, `CREDIT_TOPUP_RETAIL`, `CREDIT_TOPUP_CARD`, `CREDIT_REFUND`, `CREDIT_BONUS`, `CREDIT_ADJUSTMENT_ADMIN`, `DEBIT_CYCLE_CHARGE`, `DEBIT_RECOVERY`, `DEBIT_ADJUSTMENT_ADMIN`, `DEBIT_REFUND_REVERSAL`), an `amount` (always positive — direction is determined by type), an `idempotency_key` (caller-provided, unique per type per wallet), a `reference` (external reference: gateway transaction ID, invoice ID, refund request ID, etc.), and an `actor` (`gateway:mpesa`, `service:bil-03`, `service:bil-04`, `user:admin_xx`, etc.).	All transactions
R-BIL-05-T-3	Every transaction is idempotent on `(wallet_id, idempotency_key)`. Repeat calls with the same idempotency key for the same wallet return the prior result without re-applying the operation. This makes scanner retries and gateway-replay safe. Idempotency keys are owned by the caller — BIL-03 uses `cycle-debit-{subscription_id}-{cycle_end_iso}`, gateways use their own transaction IDs, BIL-04 uses `dunning-recovery-{subscription_id}-{recovery_at_iso}`.	Idempotency
R-BIL-05-T-4	Transactions reference the wallet row via FK. The wallet row's `balance`, `total_credits_lifetime`, `total_debits_lifetime` are updated atomically in the same DB transaction as the transaction row insert. A `SELECT FOR UPDATE` on the wallet row at the start of the operation ensures serialization.	Atomicity
R-BIL-05-T-5	Transaction rows are retained P10Y for audit/regulatory compliance. The `wallet_transaction` table is monthly partitioned per FOUNDATION_DB.md. No hard delete is performed.	Retention

ID	Rule	Triggered on
R-BIL-05-T-6	A transaction's `posted_at` timestamp is the moment the transaction commits to the database. External gateway transactions may carry an additional `external_timestamp` (when the gateway recorded the transaction on its side); this is captured but does not affect `posted_at`. Used for reconciliation.	Timestamps

Debit rules

ID	Rule	Triggered on
R-BIL-05-D-1	Debit operations are called by other billing modules via REST. Callers: BIL-03 (`DEBIT_CYCLE_CHARGE` at cycle boundary), BIL-04 (`DEBIT_RECOVERY` during PREPAID auto-recovery from dunning). Admin debits (`DEBIT_ADJUSTMENT_ADMIN`) are called from the admin console. BIL-05 does NOT initiate debits on its own — it is a reactive ledger.	All debits
R-BIL-05-D-2	A debit operation: (a) locks the wallet row (`SELECT FOR UPDATE`), (b) checks `status = ACTIVE` (if not, return `WALLET_NOT_ACTIVE`), (c) checks `balance >= amount` (if not, return `INSUFFICIENT_BALANCE` with current balance and shortfall in response), (d) inserts the `wallet_transaction` row, (e) updates the wallet row's `balance` and `total_debits_lifetime`, (f) emits `WalletDebited` Kafka event via outbox pattern, (g) commits.	Debit flow
R-BIL-05-D-3	Debit responses include: `success`, `wallet_balance_before`, `wallet_balance_after`, `transaction_id` (the wallet_transaction row's ID), `posted_at`. On failure: `failure_code` (`INSUFFICIENT_BALANCE`, `WALLET_NOT_ACTIVE`, `WALLET_NOT_FOUND`, etc.), `failure_detail`.	Debit response
R-BIL-05-D-4	Debit rejections do NOT count as transactions — no row is inserted on failure (only the call attempt is logged in application logs). The caller (BIL-03 or BIL-04) is expected to handle the rejection: fire `CyclePaymentMissed` (BIL-03), wait for next scanner cycle (BIL-04).	Failure handling

Credit / topup rules

ID	Rule	Triggered on
R-BIL-05-C-1	Topup operations are called by gateway adapters (mobile money, bank, retail, card) when an external payment provider confirms a successful payment. Each gateway adapter translates the provider's event format into the uniform topup-credit call. The call includes: `wallet_id` (resolved via `subscription_id` lookup), `amount`, `currency` (must match wallet currency), `gateway_reference` (the provider's unique transaction ID, used as `idempotency_key`), `gateway_type` (determines transaction `type` enum value), `external_timestamp`.	Topup credit
R-BIL-05-C-2	A topup credit: (a) resolves the wallet from the supplied identifier (Subscription ID or customer mobile number — gateway adapters handle the resolution), (b) locks the wallet row (`SELECT FOR UPDATE`), (c) checks `status = ACTIVE` (if `SUSPENDED`, the topup is held in a `pending_credits` queue for operator review; if `CLOSED`, the topup is rejected with `WALLET_CLOSED`), (d) checks currency match (rejects on mismatch), (e) inserts the `wallet_transaction` row, (f) updates the wallet row's `balance` and `total_credits_lifetime`, (g) emits `WalletToppedUp` Kafka event via outbox, (h) commits.	Credit flow

ID	Rule	Triggered on
R-BIL-05-C-3	Other credit types: `CREDIT_REFUND` (from BIL-02 or termination workflow — refund of a previous cycle charge or proration credit), `CREDIT_BONUS` (from BIL-CFG-01 promotional events or operator campaigns), `CREDIT_ADJUSTMENT_ADMIN` (operator manual correction). All follow the same credit flow as topups but with different transaction `type` and `actor` values. They emit `WalletCredited` (a more generic event) rather than `WalletToppedUp` so BIL-04's auto-recovery listener filters only on actual topups (BIL-04 doesn't try to auto-recover on an admin adjustment, because that's an out-of-band correction, not a customer payment).	Credit type distinctions
R-BIL-05-C-4	Customer-facing topup confirmation: every successful topup credit emits `WalletToppedUp` which the `notification` module consumes to send a "topup received" SMS / email / push. The customer experience is: pay via M-Pesa → receive confirmation SMS within seconds.	Customer notification
R-BIL-05-C-5	A topup may arrive for a Subscription in dunning. BIL-05 still credits the wallet (the wallet itself is independent of dunning state) and emits `WalletToppedUp`. BIL-04 consumes the event and decides if the new balance is sufficient to recover the cycle, per R-BIL-04-R-8. BIL-05 does not know or care about the Subscription's dunning state.	Dunning-state independence

Gateway integration rules

ID	Rule	Triggered on
R-BIL-05-G-1	Each external topup channel (M-Pesa for KE, MTN Mobile Money for UG, Vodacom M-Pesa for TZ, bank transfer rails, retail agent network, card processor) has a dedicated **gateway adapter** in BIL-05. The adapter handles: provider-specific webhook/callback parsing, signature/HMAC verification, idempotency-key extraction from provider transaction ID, wallet resolution (provider gives mobile number → BIL-05 looks up which Subscription's wallet to credit), and translation into the uniform topup-credit call.	Gateway adapter
R-BIL-05-G-2	Wallet resolution by mobile number: a customer's mobile number (provided in the gateway payload) may map to multiple PREPAID Subscriptions (e.g., one customer with internet + TV at the same address might have two Subscriptions). BIL-05 resolves to a specific Subscription via a `wallet_lookup_alias` table that maps mobile number → Subscription ID. Customers register their topup mobile number at activation (default: the primary contact mobile number from the CRM record). Customers may register multiple aliases (one per Subscription) for clarity. If no alias matches: the topup is held in `pending_credits` for operator review (rare case, indicates registration gap).	Mobile-number-to-wallet resolution
R-BIL-05-G-3	Gateway adapters are STATELESS — they receive an incoming provider event, translate, and call BIL-05's internal credit endpoint. The actual idempotency and ledger writing happens in BIL-05's service layer. Adapters do not maintain their own transaction history.	Adapter design
R-BIL-05-G-4	Gateway adapter failures (provider HMAC mismatch, payload malformed, wallet not resolved): the gateway adapter rejects the provider's webhook with a 4xx response so the provider retries. The original provider transaction is preserved on their side; once the issue is resolved (alias added, signature fixed, etc.), the provider's retry succeeds.	Adapter failure mode

ID	Rule	Triggered on
R-BIL-05-G-5	Reconciliation: BIL-05 runs a nightly job (per gateway) that fetches a transaction summary from each provider (total credited per provider per day) and compares to BIL-05's recorded credits per gateway type per day. Discrepancies are logged in a `gateway_reconciliation` table and flagged for operator review. Per-provider reconciliation API support is required (most major providers offer this).	Daily reconciliation

State interaction rules

ID	Rule	Triggered on
R-BIL-05-S-1	Wallet creation: when SUB-WF-ACTIVATE-01 activates a PREPAID Subscription, the activation workflow's billing call to BIL-01 (with `ACTIVATION_INTENT`) triggers BIL-01 to call BIL-05 `POST /api/wallets` creating the wallet row with `status = ACTIVE`, `balance = 0`. The activation workflow may also include an initial topup if the customer paid the activation fee + first cycle via the activation channel (e.g., paid via M-Pesa at signup) — that topup is processed as a normal credit operation.	Activation
R-BIL-05-S-2	Wallet closure: when SUB-WF-TERMINATE-01 terminates a PREPAID Subscription, the termination workflow's billing call to BIL-01 (with `TERMINATION_INTENT`) triggers BIL-01 to: (a) compute owed amounts (unpaid ETF, equipment write-off, etc.); (b) call BIL-05 to debit the wallet for owed amounts via `DEBIT_REFUND_REVERSAL` or `DEBIT_ADJUSTMENT_ADMIN`; (c) determine remaining wallet balance after settlement; (d) if balance > 0, initiate refund-out per R-BIL-05-S-7..S-10; (e) call BIL-05 `POST /api/wallets/{id}/close` which sets `status = CLOSED` and disallows any further credits or debits. The wallet row and its full transaction history are retained P10Y per R-BIL-05-T-5.	Termination
R-BIL-05-S-7	Refund channel selection at wallet closure (and during mid-subscription refunds): the default refund channel is **same-channel-as-most-recent-topup** — if the customer's last 3 successful topups were via M-Pesa, the refund goes to the M-Pesa number on file. Rationale: regulatory compliance (anti-money-laundering — refunds should return to the source of funds when possible) and customer convenience. If the most-recent-topup channel cannot accept refunds (e.g., retail agent cash topup with no return channel), the next-most-recent channel is tried. Final fallback: customer's registered bank account if available, else manual operator handling via the admin console.	Refund channel
R-BIL-05-S-8	Refund-out execution: refund-out is NOT executed by BIL-05 directly. Instead, BIL-05 records the intent by inserting a `wallet_transaction` row of type `CREDIT_REFUND` with `direction = CREDIT` and a paired `DEBIT_REFUND_REVERSAL` row that zeros out the wallet balance, then **publishes a `WalletRefundInitiated` Kafka event** consumed by the payout module (separate, leverages existing payment-gateway infrastructure used elsewhere). The payout module handles the actual outbound transfer (M-Pesa B2C, bank transfer, etc.) asynchronously and emits `WalletRefundCompleted` or `WalletRefundFailed` events back, **which BIL-05 consumes** to update the refund status in the admin console for any operator follow-up. So BIL-05 is the source-of-truth for refund-initiation but does not perform the outbound payment itself.	Refund-out execution

ID	Rule	Triggered on
R-BIL-05-S-9	Refund SLA: refund-out target SLA is **within 2 business days** for mobile-money channels (provider rails typically settle T+1), **within 5 business days** for bank transfers. Refunds requiring manual operator intervention (cash refund at retail agent, unusual amounts requiring fraud review) target 10 business days. SLAs are tracked via the `WalletRefundInitiated` → `WalletRefundCompleted` Kafka event pair timing.	Refund SLA
R-BIL-05-S-10	Customer notification of refund: every `WalletRefundInitiated` event is consumed by the `notification` module which dispatches an SMS + email to the customer: "A refund of X KES has been initiated to [channel last-4-digits]. Expected to arrive within [SLA] business days. Reference: [refund_id]." A second notification fires on `WalletRefundCompleted` confirming arrival, or `WalletRefundFailed` instructing the customer to contact support.	Refund notification
R-BIL-05-S-3	Wallet suspend: when fraud investigation, regulatory hold, or other operator-driven exception requires it, an admin may call `POST /api/wallets/{id}/suspend` (role `WALLET_ADMIN`). The wallet status moves to `SUSPENDED`. Any subsequent debit or credit attempts are rejected with `WALLET_NOT_ACTIVE`. Topups that arrive while suspended are queued in `pending_credits` for operator review (not lost — see R-BIL-05-C-2). The wallet can be unsuspended via `POST /api/wallets/{id}/unsuspend`.	Operator suspend
R-BIL-05-S-4	Wallet behavior during Subscription voluntary pause (SUB-WF-PAUSE-01): the wallet stays `ACTIVE`. No new cycle debits will occur during pause (BIL-03 doesn't fire `CycleClosed` on paused Subscriptions per R-BIL-03-S-1 / S-2). The customer may still top up during pause if they wish (e.g., to ensure balance for when service resumes). On resume, the next cycle close will debit the wallet normally.	Voluntary pause coexistence
R-BIL-05-S-5	Wallet behavior during Subscription non-payment suspension (SUB-WF-SUSPEND-NP-01): the wallet stays `ACTIVE`. The whole point of the suspended state in PREPAID dunning is to let the customer top up to recover. The topup → `WalletToppedUp` → BIL-04 recovery flow handles the rest (R-BIL-04-R-8).	Dunning suspension coexistence
R-BIL-05-S-6	Wallet behavior during Subscription PENDING_* (mid-workflow): no debits or credits are blocked at the wallet level — the wallet is independent of Subscription workflow state. A PENDING_UPGRADE Subscription may receive a topup or have a recovery debit during the in-flight workflow; the wallet operations succeed normally. Workflows that affect the wallet (activate, terminate) coordinate their own calls to BIL-05.	PENDING_* coexistence

3. Module owner and impacted modules

Role	Module	What it does in this process
Owner	New `billing-wallet` module	Owns the wallet entity, the transaction ledger, the credit/debit operations, the gateway adapters (per provider), the wallet admin REST endpoints, the `WalletToppedUp` and `WalletCredited` and `WalletDebited` Kafka events.
Impacted (callers via REST)	BIL-03 (cycle-debit at cycle boundary), BIL-04 (recovery debit + balance query during PREPAID dunning recovery), BIL-01 (orchestrates activation/termination calls — creating/closing wallet on Subscription lifecycle), admin console (manual operations), gateway webhooks (M-Pesa, MTN MM, banks, retail agent, card processor)	Receive synchronous REST calls from BIL-05 / send incoming webhooks.

Role	Module	What it does in this process
Impacted (downstream consumers via Kafka)	BIL-04 (consumes `WalletToppedUp` for PREPAID auto-recovery per R-BIL-04-R-8), BIL-03 (consumes `WalletToppedUp` indirectly — BIL-04 calls BIL-03's `advance-cycle-on-recovery` endpoint), `notification` module (DD_NOT-01, planned, for topup confirmations), CVM, analytics	Downstream modules consume wallet events.
Frontend	Admin Wallet Console (operator view of wallets, drill-in transaction history, manual credit/debit, suspend/unsuspend); Customer Portal Wallet (balance, transaction history, topup channels)	Read + admin via BIL-05 REST.
Engines	No Drools. Wallet operations are deterministic ledger writes.	—

4. Foundation references

DB → `FOUNDATION_DB.md` · Kafka → `FOUNDATION_KAFKA.md` · Auth → `FOUNDATION_AUTH.md`. Framework: SUB-WF-FRAMEWORK-01 (BIL-05 follows the synchronous service-class pattern; gateway adapters follow REST-handler conventions documented there).

Technical design

Module placement

Layer	Where
Frontend	Admin Wallet Console (operator view), Customer Portal wallet panel.
Backend	New `billing-wallet` module on cluster C (revenue cluster, alongside `billing`, `subscription`, `dunning`, `cycle-close`).
Rules	None (pure ledger logic).
Events consumed	2 events from payout module: `sophix.payout.wallet.refundcompleted`, `sophix.payout.wallet.refundfailed` (consumed to update refund status in admin console; refund SLA tracking)
Events emitted	4 events: `WalletToppedUp` (specifically topup credits, the trigger for BIL-04 recovery), `WalletCredited` (broader credit event covering all non-topup credits), `WalletDebited`, `WalletRefundInitiated` (consumed by payout module per R-BIL-05-S-8).

Wallet lifecycle and operation flow

[Embedded image omitted: Wallet operations and lifecycle]

Topup flow from external gateway to BIL-04 recovery

[Embedded image omitted: Topup flow]

Codebase

Java/Spring Boot layout

```
modules/billing-wallet/
  src/main/java/com/sophix/billing/wallet/
    web/rest/
      WalletResource.java                       ← admin + read endpoints
      WalletTransactionResource.java            ← transaction history
      dto/...
    web/gateway/
      MPesaWebhookController.java               ← M-Pesa STK Push callback handler (KE)
      MtnMobileMoneyWebhookController.java      ← MTN Mobile Money callback (UG)
      VodacomMpesaWebhookController.java        ← Vodacom M-Pesa (TZ)
      BankTransferWebhookController.java        ← bank rails callback
      RetailAgentWebhookController.java         ← retail agent topup
      CardProcessorWebhookController.java       ← online card payment
    service/
      WalletService.java                        ← core debit/credit logic
      WalletCreationService.java                ← wallet creation on activation
      WalletClosureService.java                 ← wallet closure on termination
      WalletAdminService.java                   ← suspend/unsuspend, admin adjustments
      WalletLookupService.java                  ← mobile-number-to-wallet resolution
```

```
    GatewayReconciliationService.java            ← nightly reconciliation per gateway
  gateway/
    MPesaGatewayAdapter.java                     ← provider-specific event translation
    MtnMobileMoneyGatewayAdapter.java
    VodacomMpesaGatewayAdapter.java
    BankTransferGatewayAdapter.java
    RetailAgentGatewayAdapter.java
    CardProcessorGatewayAdapter.java
    GatewayAdapter.java                          ← shared interface
  domain/
    Wallet.java                                  ← entity
    WalletStatus.java                            ← enum ACTIVE | SUSPENDED | CLOSED
    WalletTransaction.java                       ← entity
    WalletTransactionType.java                   ← enum (12 types per R-BIL-05-T-2)
    WalletLookupAlias.java                       ← mobile-number alias entity
    PendingCredit.java                           ← held topups awaiting operator review
    GatewayReconciliation.java
  repository/
    WalletRepository.java
    WalletTransactionRepository.java
    WalletLookupAliasRepository.java
    PendingCreditRepository.java
    GatewayReconciliationRepository.java
  scheduler/
    GatewayReconciliationScanner.java            ← nightly @Scheduled per gateway
  event/
    WalletToppedUpEventPublisher.java
    WalletCreditedEventPublisher.java
    WalletDebitedEventPublisher.java
    WalletRefundInitiatedEventPublisher.java      ← new, emitted on refund-out initiation
  kafka/
    WalletRefundCompletedConsumer.java            ← consumes payout module success event
    WalletRefundFailedConsumer.java               ← consumes payout module failure event
src/main/resources/
  config/liquibase/changelog/billing-wallet/
    001_wallet.xml
    002_wallet_transaction.xml
    003_wallet_lookup_alias.xml
    004_pending_credit.xml
    005_gateway_reconciliation.xml
    006_refund_request.xml
```

Laravel/PHP parallel layout

```
sophix/billing-wallet/
  src/Http/Controllers/
    WalletController.php
    WalletTransactionController.php
  src/Http/Gateway/
    MPesaWebhookController.php
    MtnMobileMoneyWebhookController.php
    VodacomMpesaWebhookController.php
    BankTransferWebhookController.php
    RetailAgentWebhookController.php
    CardProcessorWebhookController.php
  src/Service/
    WalletService.php
    WalletCreationService.php
    WalletClosureService.php
    WalletAdminService.php
    WalletLookupService.php
    GatewayReconciliationService.php
  src/Gateway/
    MPesaGatewayAdapter.php
    MtnMobileMoneyGatewayAdapter.php
    VodacomMpesaGatewayAdapter.php
    BankTransferGatewayAdapter.php
    RetailAgentGatewayAdapter.php
    CardProcessorGatewayAdapter.php
    GatewayAdapterInterface.php
  src/Domain/
    Wallet.php
    WalletStatus.php
    WalletTransaction.php
    WalletTransactionType.php
    WalletLookupAlias.php
    PendingCredit.php
    GatewayReconciliation.php
  src/Repository/
    WalletRepository.php
    WalletTransactionRepository.php
    WalletLookupAliasRepository.php
    PendingCreditRepository.php
```

```
  GatewayReconciliationRepository.php
src/Console/
  GatewayReconciliationCommand.php                ← scheduled nightly
src/Event/
  WalletToppedUpEventPublisher.php
  WalletCreditedEventPublisher.php
  WalletDebitedEventPublisher.php
  WalletRefundInitiatedEventPublisher.php          ← new, emitted on refund-out initiation
src/Kafka/
  WalletRefundCompletedListener.php                ← consumes payout module success event
  WalletRefundFailedListener.php                   ← consumes payout module failure event
src/Providers/
  BillingWalletServiceProvider.php
routes/
  api.php
database/migrations/
  2026_09_01_create_wallet_table.php
  2026_09_01_create_wallet_transaction_table.php
  2026_09_01_create_wallet_lookup_alias_table.php
  2026_09_01_create_pending_credit_table.php
  2026_09_01_create_gateway_reconciliation_table.php
  2026_09_01_create_refund_request_table.php
```

Foundation-specific configurations

Foundation	Configuration
DB	New tables on cluster C: `wallet` (one row per PREPAID Subscription, ~150-200k rows for Wananchi target), `wallet_transaction` (monthly partitioned, P10Y retention — major-volume table), `wallet_lookup_alias` (mobile-number-to-wallet mapping), `pending_credit` (topups awaiting operator review for SUSPENDED wallets), `gateway_reconciliation` (nightly reconciliation results), `refund_request` (refund-out tracking per R-BIL-05-S-8). `_audit` shadow tables on `wallet`, `wallet_lookup_alias`, `refund_request`; `wallet_transaction` is immutable so no shadow needed.
Drools	None.
Kafka	Emits 4 topics, consumes 2 topics (refund completion from payout module). All on cluster C Kafka brokers per FOUNDATION_KAFKA.md.
Auth	Roles: `WALLET_ADMIN` (manual suspend/unsuspend/credit/debit adjustments), `WALLET_OPERATOR` (read-only investigation, pending_credit review), `BILLING_INTERNAL` (BIL-01 / BIL-03 / BIL-04 callers), `GATEWAY_INTERNAL` (gateway webhook authentication).

Frontend

Screen	Purpose
Admin Wallet Console — Dashboard	Counts: active wallets, suspended wallets, today's topup volume per gateway, today's debit volume, reconciliation status
Admin Wallet Console — Wallet Drill-in	One Subscription's wallet: current balance, status, lookup aliases, full transaction history (paginated), suspend/unsuspend / manual credit / manual debit actions
Admin Wallet Console — Pending Credits Queue	Topups held due to wallet suspended or alias not found, with operator actions: route to wallet / refund to gateway / investigate
Admin Wallet Console — Gateway Reconciliation	Per-gateway daily reconciliation results, discrepancies flagged for follow-up
Customer Portal — Wallet	Balance, recent transactions (paginated), topup channels (per operator: M-Pesa instructions for KE, MTN MM for UG, etc.), suggested topup amount (= upcoming cycle charge minus current balance if positive)

Data model

`wallet`

One row per PREPAID Subscription.

Column	Type	Notes
`id`	TEXT PK	ULID
`subscription_id`	TEXT NOT NULL UNIQUE	One wallet per Subscription
`customer_id`	TEXT NOT NULL	Denormalized for query convenience
`operator_code`	TEXT NOT NULL	`WANANCHI_KE` / `WANANCHI_UG` / `WANANCHI_TZ`

Column	Type	Notes
currency	TEXT NOT NULL	KES / UGX / TZS (XOF reserved for V3.2 Yas SN PREPAID)
status	TEXT NOT NULL	ACTIVE / SUSPENDED / CLOSED
balance	NUMERIC(15,2) NOT NULL DEFAULT 0	Current balance (sum of credits - sum of debits)
total_credits_lifetime	NUMERIC(15,2) NOT NULL DEFAULT 0	Sum of all credits ever
total_debits_lifetime	NUMERIC(15,2) NOT NULL DEFAULT 0	Sum of all debits ever
created_at	TIMESTAMPTZ NOT NULL	When wallet was created
last_transaction_at	TIMESTAMPTZ NULL	Most recent transaction (for activity heuristics)
suspended_at	TIMESTAMPTZ NULL	When suspended (if applicable)
suspended_reason	TEXT NULL	Reason code
closed_at	TIMESTAMPTZ NULL	When closed

Indexes:

- (subscription_id) UNIQUE
- (customer_id) for customer-wide wallet queries
- (status, operator_code) for operator-scoped admin queries

_audit shadow table per FOUNDATION_DB.md.

wallet_transaction

Immutable ledger entry. Monthly partitioned.

Column	Type	Notes
id	TEXT PK	ULID, also serves as transaction_id returned to callers
wallet_id	TEXT NOT NULL	FK to wallet.id
subscription_id	TEXT NOT NULL	Denormalized for query
type	TEXT NOT NULL	12 enum values per R-BIL-05-T-2
amount	NUMERIC(15,2) NOT NULL	Always positive
currency	TEXT NOT NULL	Matches wallet currency
direction	TEXT NOT NULL	CREDIT or DEBIT (derived from type but stored for query)
idempotency_key	TEXT NOT NULL	Unique per (wallet_id, idempotency_key)
reference	TEXT NULL	Gateway transaction ID / invoice ID / etc.
gateway_type	TEXT NULL	For topups: MPESA_KE / MTN_MOMO_UG / VODACOM_MPESA_TZ / BANK_TRANSFER / RETAIL_AGENT / CARD
external_timestamp	TIMESTAMPTZ NULL	Gateway-side timestamp (when applicable)
actor	TEXT NOT NULL	Who initiated (service / gateway / user)
wallet_balance_before	NUMERIC(15,2) NOT NULL	Balance snapshot before this transaction
wallet_balance_after	NUMERIC(15,2) NOT NULL	Balance snapshot after
posted_at	TIMESTAMPTZ NOT NULL	When this row committed
metadata	JSONB NULL	Provider-specific extras (mobile number used, receipt number, etc.)

Indexes:

- (wallet_id, posted_at DESC) for transaction history queries
- (wallet_id, idempotency_key) UNIQUE for idempotency enforcement
- (type, posted_at) for type-scoped reporting
- (gateway_type, posted_at) for gateway reconciliation

P10Y retention. Monthly partitions for query performance and archive efficiency.

wallet_lookup_alias

Maps external identifiers (typically mobile numbers) to wallets. Topup gateways provide the mobile number; BIL-05 resolves to the right wallet.

Column	Type	Notes
`id`	TEXT PK	ULID
`alias_type`	TEXT NOT NULL	`MOBILE_NUMBER` (V3) — `CARD_TOKEN`, `BANK_ACCOUNT` reserved for V3.2
`alias_value`	TEXT NOT NULL	The actual mobile number / token / account (normalized format)
`wallet_id`	TEXT NOT NULL	FK to `wallet.id`
`subscription_id`	TEXT NOT NULL	Denormalized
`is_primary`	BOOLEAN NOT NULL DEFAULT TRUE	Customers may register multiple aliases; the primary is used for default routing
`registered_at`	TIMESTAMPTZ NOT NULL	
`registered_by`	TEXT NOT NULL	`customer:self_service` / `agent:xxx` / `system:activation`
`verified`	BOOLEAN NOT NULL DEFAULT FALSE	Set after the first successful topup via this alias
`last_used_at`	TIMESTAMPTZ NULL	

Indexes:

- `(alias_type, alias_value)` UNIQUE per `(alias_type, alias_value, operator_code)` — same mobile number could in theory exist across operators; scope ensures correctness
- `(wallet_id)` for wallet-side queries

`_audit` shadow table.

`pending_credit`

Held topups awaiting operator review (e.g., wallet suspended or alias not found).

Column	Type	Notes
`id`	TEXT PK	ULID
`gateway_type`	TEXT NOT NULL	Source gateway
`gateway_reference`	TEXT NOT NULL	Provider's transaction ID
`amount`	NUMERIC(15,2) NOT NULL	
`currency`	TEXT NOT NULL	
`claimed_alias`	TEXT NULL	The mobile number / identifier the gateway provided
`external_timestamp`	TIMESTAMPTZ NOT NULL	
`hold_reason`	TEXT NOT NULL	`WALLET_SUSPENDED` / `ALIAS_NOT_FOUND` / `CURRENCY_MISMATCH` / etc.
`status`	TEXT NOT NULL	`HELD` / `RESOLVED_CREDITED` / `RESOLVED_REFUNDED` / `RESOLVED_DISCARDED`
`created_at`	TIMESTAMPTZ NOT NULL	
`resolved_at`	TIMESTAMPTZ NULL	
`resolved_by`	TEXT NULL	Operator who resolved
`resolution_notes`	TEXT NULL	

P10Y retention.

`gateway_reconciliation`

Nightly reconciliation results.

Column	Type	Notes
`id`	TEXT PK	ULID
`gateway_type`	TEXT NOT NULL	
`reconciliation_date`	DATE NOT NULL	The day being reconciled
`provider_total_credited`	NUMERIC(15,2)	What the provider says was credited
`bil05_total_credited`	NUMERIC(15,2)	What BIL-05 recorded

Column	Type	Notes
provider_transaction_count	INTEGER	
bil05_transaction_count	INTEGER	
discrepancy_amount	NUMERIC(15,2)	Provider - BIL-05 (positive = BIL-05 missing transactions)
discrepancy_transaction_count	INTEGER	
status	TEXT NOT NULL	MATCHED / MINOR_DISCREPANCY / MAJOR_DISCREPANCY / PROVIDER_API_FAILED
resolved_at	TIMESTAMPTZ NULL	
resolved_by	TEXT NULL	

P10Y retention.

refund_request

Tracks outbound refund requests initiated by BIL-05 (per R-BIL-05-S-8). Linked to the wallet_transaction credit/debit pair that recorded the refund intent.

Column	Type	Notes
id	TEXT PK	ULID (refund_*), used as refundId in events
wallet_id	TEXT NOT NULL	FK to wallet.id
subscription_id	TEXT NOT NULL	Denormalized
customer_id	TEXT NOT NULL	Denormalized
amount	NUMERIC(15,2) NOT NULL	Refund amount
currency	TEXT NOT NULL	
refund_channel	TEXT NOT NULL	MPESA_KE / MTN_MOMO_UG / VODACOM_MPESA_TZ / BANK_TRANSFER / etc.
refund_destination	TEXT NOT NULL	Masked mobile number / account (last 4 digits exposed in events)
refund_reason	TEXT NOT NULL	SUBSCRIPTION_TERMINATION / INVOICE_CREDIT / OVERPAYMENT_CORRECTION / etc.
expected_sla_business_days	INTEGER NOT NULL	Per R-BIL-05-S-9
status	TEXT NOT NULL	INITIATED / COMPLETED / FAILED / REQUIRES_MANUAL
credit_transaction_id	TEXT NOT NULL	The CREDIT_REFUND wallet_transaction row
debit_transaction_id	TEXT NOT NULL	The paired DEBIT_REFUND_REVERSAL row
payout_provider_reference	TEXT NULL	Provider's reference (M-Pesa B2C ID, bank transfer ref) — set on completion
failure_code	TEXT NULL	Set on failure
failure_detail	TEXT NULL	
initiated_at	TIMESTAMPTZ NOT NULL	
completed_at	TIMESTAMPTZ NULL	
failed_at	TIMESTAMPTZ NULL	

Indexes:

- (wallet_id, initiated_at DESC) for customer refund history
- (status, initiated_at) for admin queue queries

P10Y retention. _audit shadow table.

Wallet service core logic

```
@Service
public class WalletService {

    @Transactional
    public DebitResult debit(String walletId, BigDecimal amount, WalletTransactionType type,
                             String idempotencyKey, String reference, String actor) {
        // Idempotency check
```

```
        Optional<WalletTransaction> prior = transactionRepo.findByWalletAndIdempotencyKey(walletId, idempotencyKey);
        if (prior.isPresent()) {
            return DebitResult.idempotentReplay(prior.get());
        }

        Wallet wallet = walletRepo.lockById(walletId);
        if (wallet == null) {
            return DebitResult.failure("WALLET_NOT_FOUND");
        }
        if (wallet.getStatus() != WalletStatus.ACTIVE) {
            return DebitResult.failure("WALLET_NOT_ACTIVE", "wallet status is " + wallet.getStatus());
        }
        if (wallet.getBalance().compareTo(amount) < 0) {
            return DebitResult.failure(
                "INSUFFICIENT_BALANCE",
                "balance " + wallet.getBalance() + " < requested " + amount
            );
        }

        BigDecimal balanceBefore = wallet.getBalance();
        BigDecimal balanceAfter = balanceBefore.subtract(amount);

        WalletTransaction txn = new WalletTransaction();
        txn.setWalletId(walletId);
        txn.setSubscriptionId(wallet.getSubscriptionId());
        txn.setType(type);
        txn.setDirection(Direction.DEBIT);
        txn.setAmount(amount);
        txn.setCurrency(wallet.getCurrency());
        txn.setIdempotencyKey(idempotencyKey);
        txn.setReference(reference);
        txn.setActor(actor);
        txn.setWalletBalanceBefore(balanceBefore);
        txn.setWalletBalanceAfter(balanceAfter);
        txn.setPostedAt(Instant.now());
        transactionRepo.save(txn);

        wallet.setBalance(balanceAfter);
        wallet.setTotalDebitsLifetime(wallet.getTotalDebitsLifetime().add(amount));
        wallet.setLastTransactionAt(Instant.now());
        walletRepo.save(wallet);

        eventPublisher.publishWalletDebited(buildWalletDebitedEvent(wallet, txn));

        return DebitResult.success(txn.getId(), balanceBefore, balanceAfter);
    }

    @Transactional
    public CreditResult credit(String walletId, BigDecimal amount, WalletTransactionType type,
                               String idempotencyKey, String reference, String gatewayType,
                               Instant externalTimestamp, String actor) {
        // Idempotency check
        Optional<WalletTransaction> prior = transactionRepo.findByWalletAndIdempotencyKey(walletId, idempotencyKey);
        if (prior.isPresent()) {
            return CreditResult.idempotentReplay(prior.get());
        }

        Wallet wallet = walletRepo.lockById(walletId);
        if (wallet == null) {
            return CreditResult.failure("WALLET_NOT_FOUND");
        }
        if (wallet.getStatus() == WalletStatus.CLOSED) {
            return CreditResult.failure("WALLET_CLOSED");
        }
        if (wallet.getStatus() == WalletStatus.SUSPENDED) {
            // Hold as pending credit per R-BIL-05-C-2
            pendingCreditService.holdForReview(wallet, amount, gatewayType, reference, externalTimestamp, "WALLET_SUSPENDE
            return CreditResult.heldPending();
        }

        BigDecimal balanceBefore = wallet.getBalance();
        BigDecimal balanceAfter = balanceBefore.add(amount);

        WalletTransaction txn = new WalletTransaction();
        txn.setWalletId(walletId);
        txn.setSubscriptionId(wallet.getSubscriptionId());
        txn.setType(type);
        txn.setDirection(Direction.CREDIT);
        txn.setAmount(amount);
        txn.setCurrency(wallet.getCurrency());
        txn.setIdempotencyKey(idempotencyKey);
        txn.setReference(reference);
```

```
            txn.setGatewayType(gatewayType);
            txn.setExternalTimestamp(externalTimestamp);
            txn.setActor(actor);
            txn.setWalletBalanceBefore(balanceBefore);
            txn.setWalletBalanceAfter(balanceAfter);
            txn.setPostedAt(Instant.now());
            transactionRepo.save(txn);

            wallet.setBalance(balanceAfter);
            wallet.setTotalCreditsLifetime(wallet.getTotalCreditsLifetime().add(amount));
            wallet.setLastTransactionAt(Instant.now());
            walletRepo.save(wallet);

            // Emit specific event per credit type
            if (isTopup(type)) {
                eventPublisher.publishWalletToppedUp(buildWalletToppedUpEvent(wallet, txn));
            } else {
                eventPublisher.publishWalletCredited(buildWalletCreditedEvent(wallet, txn));
            }

            return CreditResult.success(txn.getId(), balanceBefore, balanceAfter);
        }
    }
```

The Laravel/PHP equivalent follows the same logic with `DB::transaction()` closure and Throwable handling, per the dual-stack pattern in SUB-WF-FRAMEWORK-01.

Events (Kafka)

`sophix.billing.wallet.toppedup`

Emitted on every successful topup credit (the 4 CREDIT_TOPUP_* types). The **single most important downstream event** in BIL-05 — BIL-04 consumes it for PREPAID auto-recovery (per R-BIL-04-R-8); notification consumes for the customer "topup received" confirmation.

```
{
  "eventType": "WalletToppedUp",
  "aggregateId": "wallet_01HZ...",
  "timestamp": "2026-09-05T11:00:01Z",
  "payload": {
    "walletId": "wallet_01HZ...",
    "subscriptionId": "sub_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WANANCHI_KE",
    "currency": "KES",
    "amount": 5000.00,
    "gatewayType": "MPESA_KE",
    "gatewayReference": "MPESA_TX_RKL3HD8FN2",
    "walletBalanceBefore": 0.00,
    "walletBalanceAfter": 5000.00,
    "transactionId": "wt_01HZ...",
    "externalTimestamp": "2026-09-05T11:00:00Z",
    "postedAt": "2026-09-05T11:00:01Z"
  }
}
```

`sophix.billing.wallet.credited`

Emitted on every non-topup credit (refund, bonus, admin adjustment). Less time-critical than `WalletToppedUp` — notification module may still surface to customer for refunds/bonuses but BIL-04 does NOT trigger auto-recovery on these (because admin adjustments and refunds are out-of-band; customer-initiated topup is the recovery signal per R-BIL-05-C-3).

```
{
  "eventType": "WalletCredited",
  "aggregateId": "wallet_01HZ...",
  "timestamp": "2026-09-10T15:30:00Z",
  "payload": {
    "walletId": "wallet_01HZ...",
    "subscriptionId": "sub_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WANANCHI_KE",
    "currency": "KES",
    "amount": 250.00,
    "creditType": "CREDIT_BONUS",
    "reference": "promo_q3_2026_loyalty",
    "walletBalanceBefore": 1500.00,
    "walletBalanceAfter": 1750.00,
    "transactionId": "wt_01HZ...",
    "postedAt": "2026-09-10T15:30:00Z"
  }
```

```
}
```

`sophix.billing.wallet.debited`

Emitted on every debit. Consumed by analytics, CVM for behavioral signals.

```
{
  "eventType": "WalletDebited",
  "aggregateId": "wallet_01HZ...",
  "timestamp": "2026-10-01T00:00:01Z",
  "payload": {
    "walletId": "wallet_01HZ...",
    "subscriptionId": "sub_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WANANCHI_KE",
    "currency": "KES",
    "amount": 4500.00,
    "debitType": "DEBIT_CYCLE_CHARGE",
    "reference": "cycle_2026_10_sub_01HZ",
    "walletBalanceBefore": 5000.00,
    "walletBalanceAfter": 500.00,
    "transactionId": "wt_01HZ...",
    "postedAt": "2026-10-01T00:00:01Z"
  }
}
```

`sophix.billing.wallet.refundinitiated`

Emitted when a refund-out is initiated (during PREPAID Subscription termination with remaining balance, or mid-subscription refund). **Consumed by the payout module** which performs the actual outbound transfer (M-Pesa B2C, bank transfer, etc.). Also consumed by `notification` module for the customer "refund initiated" SMS/email per R-BIL-05-S-10.

```
{
  "eventType": "WalletRefundInitiated",
  "aggregateId": "wallet_01HZ...",
  "timestamp": "2026-12-15T10:00:00Z",
  "payload": {
    "walletId": "wallet_01HZ...",
    "subscriptionId": "sub_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WANANCHI_KE",
    "currency": "KES",
    "refundAmount": 1200.00,
    "refundId": "refund_01HZ...",
    "refundChannel": "MPESA_KE",
    "refundDestination": "+254712****345",
    "refundReason": "SUBSCRIPTION_TERMINATION",
    "expectedSlaBusinessDays": 2,
    "creditTransactionId": "wt_credit_01HZ...",
    "debitTransactionId": "wt_debit_01HZ...",
    "initiatedAt": "2026-12-15T10:00:00Z"
  }
}
```

Consumed events from payout module

`sophix.payout.wallet.refundcompleted` — emitted by the payout module when the outbound refund successfully arrives at the customer's channel (M-Pesa B2C confirmation, bank transfer completion). BIL-05 consumes this to update the refund status in the admin console (status = `COMPLETED`) and trigger the `WalletRefundCompleted` customer confirmation notification per R-BIL-05-S-10.

```
{
  "eventType": "WalletRefundCompleted",
  "aggregateId": "refund_01HZ...",
  "timestamp": "2026-12-16T14:30:00Z",
  "payload": {
    "refundId": "refund_01HZ...",
    "walletId": "wallet_01HZ...",
    "subscriptionId": "sub_01HZ...",
    "refundAmount": 1200.00,
    "refundChannel": "MPESA_KE",
    "payoutProviderReference": "MPESA_B2C_RKL5JD9HK",
    "completedAt": "2026-12-16T14:30:00Z"
  }
}
```

`sophix.payout.wallet.refundfailed` — emitted by the payout module when the outbound refund fails (provider rejection, invalid destination, insufficient float at provider, etc.). BIL-05 consumes this to flag the refund for operator manual handling (admin console queue) and triggers the `WalletRefundFailed` customer notification ("contact support").

```
{
```

```
  "eventType": "WalletRefundFailed",
  "aggregateId": "refund_01HZ...",
  "timestamp": "2026-12-16T15:00:00Z",
  "payload": {
    "refundId": "refund_01HZ...",
    "walletId": "wallet_01HZ...",
    "subscriptionId": "sub_01HZ...",
    "refundAmount": 1200.00,
    "refundChannel": "MPESA_KE",
    "failureCode": "INVALID_DESTINATION",
    "failureDetail": "Mobile number not registered with M-Pesa",
    "requiresOperatorAttention": true,
    "failedAt": "2026-12-16T15:00:00Z"
  }
}
```

APIs (in this module)

Read endpoints (role `WALLET_OPERATOR` or `WALLET_ADMIN` for admin views, customer-self-service for own wallet)

```
GET /api/wallets/{subscriptionId}                          ← wallet by subscription
GET /api/wallets/{subscriptionId}/balance                  ← lightweight balance-only
GET /api/wallets/{subscriptionId}/transactions?limit=50    ← paginated transaction history
GET /api/wallets/{subscriptionId}/aliases                  ← list aliases registered
GET /api/wallets?operator={code}&status={status}&limit=50  ← admin listing
```

Internal endpoints (role `BILLING_INTERNAL`, called by BIL-01 / BIL-03 / BIL-04)

```
POST /api/wallets                                          ← create wallet on PREPAID activation (called by BIL-01)
POST /api/wallets/{id}/close                               ← close on termination (called by BIL-01)
POST /api/wallets/{id}/debit                               ← debit operation (called by BIL-03 cycle charge, BIL-04 recov
POST /api/wallets/{id}/credit                              ← non-topup credits (refund from BIL-02, etc.)
```

Debit request body:

```
{
  "amount": 4500.00,
  "type": "DEBIT_CYCLE_CHARGE",
  "idempotencyKey": "cycle-debit-sub_01HZ-2026-09-30T23:59:59Z",
  "reference": "cycle_2026_10_sub_01HZ",
  "actor": "service:bil-03"
}
```

Gateway webhook endpoints (role `GATEWAY_INTERNAL`)

```
POST /api/gateway/mpesa/callback                           ← M-Pesa STK Push and C2B callbacks
POST /api/gateway/mtn-momo/callback                        ← MTN Mobile Money
POST /api/gateway/vodacom-mpesa/callback                   ← Vodacom M-Pesa TZ
POST /api/gateway/bank/callback                            ← bank rails
POST /api/gateway/retail-agent/callback                    ← retail agent network
POST /api/gateway/card/callback                            ← card processor
```

Each gateway endpoint has its own auth scheme (HMAC, OAuth, IP whitelist — provider-specific) and translates incoming events into the uniform credit operation.

Customer-self-service endpoints (role customer-authenticated)

```
GET /api/customers/me/wallets                              ← all wallets for the logged-in customer
GET /api/wallets/{subscriptionId}/topup-channels           ← which gateways are available for this wallet (per operator)
POST /api/wallets/{subscriptionId}/aliases                 ← register / update mobile-number alias
DELETE /api/wallets/{subscriptionId}/aliases/{aliasId}     ← remove a registered alias
```

Admin endpoints (role `WALLET_ADMIN`)

```
POST /api/wallets/{id}/suspend                             ← suspend with reason
POST /api/wallets/{id}/unsuspend                           ← unsuspend
POST /api/wallets/{id}/credit-adjustment                   ← admin manual credit (audited)
POST /api/wallets/{id}/debit-adjustment                    ← admin manual debit (audited)
GET  /api/pending-credits?status=HELD&limit=50             ← review queue
POST /api/pending-credits/{id}/route-to-wallet             ← route a pending credit to a wallet
POST /api/pending-credits/{id}/refund-to-gateway           ← refund a pending credit back to source
POST /api/pending-credits/{id}/discard                     ← discard (rare, audited)
GET  /api/gateway-reconciliations?date=YYYY-MM-DD          ← reconciliation results
POST /api/gateway-reconciliations/{id}/resolve             ← mark as resolved with notes
```

Cross-DD coordination

Touched	What BIL-05 owns	What the other DD owns
SUB-WF-FRAMEWORK-01	Follows the synchronous service-class pattern.	Owns the framework.

Touched	What BIL-05 owns	What the other DD owns
SUB-WF-ACTIVATE-01 (PREPAID Subscription activation)	Wallet creation via `POST /api/wallets` is called from BIL-01 during the activation workflow's billing-call (which BIL-01 orchestrates from the workflow's billing envelope).	Owns the activation workflow and the wallet-creation trigger via BIL-01.
SUB-WF-TERMINATE-01 (PREPAID Subscription termination)	Wallet closure via `POST /api/wallets/{id}/close` is called from BIL-01 during the termination workflow. Final balance settlement (debit owed amounts, credit refundable balance via BIL-02's refund path) is orchestrated by BIL-01.	Owns the termination workflow.
BIL-01 (Charging Engine)	Receives wallet-create / wallet-close calls from BIL-01 during PREPAID Subscription lifecycle.	Owns charging engine; orchestrates wallet creation at PREPAID activation and wallet closure at termination.
BIL-02 (Invoicing)	Receives `CREDIT_REFUND` calls from BIL-02 when an invoice issues a refund-to-wallet (e.g., proration credit on package downgrade).	Owns invoice generation; decides whether refunds go to wallet or external channel.
BIL-03 (Cycle Close)	Receives `DEBIT_CYCLE_CHARGE` calls from BIL-03 at each PREPAID cycle boundary. Receives balance queries from BIL-03 for daily wallet-low pre-notice scan (R-BIL-03-W-5).	Owns cycle boundary detection; decides when to debit the wallet.
BIL-04 (Dunning)	Emits `WalletToppedUp` consumed by BIL-04 for PREPAID auto-recovery (R-BIL-04-R-8). BIL-04 calls BIL-05 for wallet balance query and recovery debit.	Owns dunning state machine; consumes `WalletToppedUp` and decides if recovery is triggered.
`notification` **module (DD_NOT-01, planned)**	Emits 3 wallet events (`WalletToppedUp` / `WalletCredited` / `WalletDebited`) with all fields a notification template needs (amount, balance after, gateway type, reference).	Consumes events. Expected customer comms: topup confirmation SMS / email, refund credit confirmation, low-balance proactive nudge (driven by BIL-03's `WalletLowBeforeCycle` event, not directly by BIL-05).
Gateway providers (M-Pesa, MTN MM, Vodacom M-Pesa, banks, retail agents, card processor)	Receives provider webhooks at gateway-specific endpoints; translates to uniform credit operation.	External services.
Payout module (DD_PAY-01, pending)	Emits `WalletRefundInitiated` consumed by payout module. Consumes `WalletRefundCompleted` and `WalletRefundFailed` from payout to update `refund_request.status`.	Owns outbound payment execution: M-Pesa B2C, MTN MoMo Disburse, bank transfer-out, etc. Tracks payout-side state separately.

Dependencies

Dependency	Owner	Status
SUB-WF-FRAMEWORK-01	subscription team	✓ DD complete (v0.9)
SUB-LM-01 (subscription_id, customer_id lookup for wallet creation)	subscription team	✓ DD complete (v0.8)
BIL-01 (orchestrates wallet creation/closure at PREPAID Subscription lifecycle; calls BIL-05)	billing team	■ DD partial (v0.8 — wallet integration not yet specified)
BIL-03 (calls BIL-05 for cycle debit and balance query)	billing team	✓ DD complete (v1.1)
BIL-04 (consumes `WalletToppedUp`; calls BIL-05 for recovery balance query + debit)	billing team	✓ DD complete (v1.4)
BIL-02 (calls BIL-05 for refund credits)	billing team	■ pending DD
FOUNDATION_DB.md	infra team	✓ Documented
FOUNDATION_KAFKA.md	infra team	✓ Documented
FOUNDATION_AUTH.md (`WALLET_ADMIN`, `WALLET_OPERATOR`, `BILLING_INTERNAL`, `GATEWAY_INTERNAL` roles)	infra team	✓ Documented
External gateway provider integrations (M-Pesa KE, MTN MM UG, Vodacom M-Pesa TZ, banks, retail, card)	infra / partnerships	■ partial — existing TZ integrations may be reusable; new contracts required for V3 Wananchi PREPAID
Payout module (`WalletRefundInitiated` consumer; emits `WalletRefundCompleted`/`Failed`)	payments infra team	■ pending DD (DD_PAY-01, V3 backlog) — leverages existing TZ payout infrastructure
DD_NOT-01 Notification Service	notification team	■ pending DD (Wave 1 backlog)

B — Current TZ

To be filled in after codebase review. No claims about what the TZ-deployed Sophix codebase does for wallet and top-up are recorded here until the codebase has been read directly.

The cartography phase (per the pending plan) will produce this content. Once the codebase is available, this section will be filled with verified statements about which wallet-and-topup capabilities exist and which do not — replacing this placeholder.

C — V3 design rationale

This DD's V3 design choices follow from the workshop requirements (section A) and the V3 plan to make PREPAID wallet the primary billing model for the majority of Wananchi customers. The design intent:

What V3 introduces

1. **Dedicated wallet module** supporting PREPAID as a **primary** billing model. V3 supports POSTPAID, PREPAID, PREPAYMENT side-by-side, with the wallet being the core data structure for PREPAID.
2. **Strict idempotency** on every credit and debit operation, keyed by `(wallet_id, idempotency_key)`. Gateway adapters extract idempotency keys from provider transaction IDs to prevent double-credit on webhook replay.
3. **Materialized balance** on the wallet row (fast O(1) read) atomically updated with each transaction. SQL sum-on-demand would not scale to the planned PREPAID volumes.
4. **Pluggable gateway adapter pattern** — M-Pesa KE, Vodacom M-Pesa TZ, MTN MM UG, banks, retail agents, card processor — each as a separate adapter implementing a shared interface. Provider-specific webhook parsing, signature verification, and idempotency extraction live in the adapter; the wallet core sees a uniform "credit-this-wallet" call.
5. `wallet_lookup_alias` **table** allowing customers to register multiple mobile numbers per wallet (e.g., husband and wife both topping up the family Subscription). Verification flow per registration.
6. `pending_credit` **queue** for topups that fail wallet resolution or arrive during suspended state — preserves money rather than losing it. Operator review queue for triage.
7. **Nightly automated** `gateway_reconciliation` per provider with operator review queue for discrepancies.
8. **3 Kafka events** — `WalletToppedUp` driving BIL-04 PREPAID auto-recovery + customer notification; `WalletCredited` for non-topup credits; `WalletDebited` for analytics. Event-driven downstream coordination replaces polling.
9. **Explicit** `SUSPENDED` **wallet state** with admin workflow and audit. Fraud holds and similar concerns are first-class, not direct-DB-flag hacks.
10. **Immutable transaction ledger** with P10Y retention, monthly partitioned. Corrections are new transactions; no editing of past rows.
11. **Per-Subscription wallet** (not per-Customer in V3 — simpler, matches per-Subscription cycle and dunning models). Multi-subscription customers have multiple wallets.
12. **Customer self-service alias registration** via portal (customer can register additional mobile numbers).
13. **Explicit** `currency` **column** on the wallet matching the Subscription's currency (KES for Wananchi KE, UGX for Wananchi UG, TZS for Wananchi TZ). Multi-currency wallets are not supported in V3.
14. **Real-time auto-recovery coordination with BIL-04** via `WalletToppedUp` events.
15. **Clean cycle integration with BIL-03** via REST at cycle boundary (BIL-03 calls BIL-05 to debit wallet for PREPAID cycle activation).

Migration approach (TZ-baseline-agnostic)

1. Deploy `billing-wallet` module with 5 tables on cluster C.
2. **Backfill phase** (one-time at V3 cutover): for each Wananchi Subscription that will be migrated to PREPAID model in V3:

- Create a wallet row with `balance = 0` (or with the customer's current outstanding credit/debit if any — operator-supplied opening balance).
- Create a `wallet_lookup_alias` row with the customer's primary mobile number.

- If the redeployed codebase carries any existing wallet history for that customer, the cartography phase will define the mapping from existing data → V3 transactions. Optionally backfill the most recent N months of transactions for customer-facing history continuity (not required for ledger correctness — balance is what matters).

3. Deploy gateway adapters for each provider. Initial provider list per operator:

- Wananchi KE: M-Pesa STK Push, M-Pesa C2B, bank transfer (KCB, Equity), retail agent network

- Wananchi UG: MTN Mobile Money, Airtel Money, bank transfer, retail agents

- Wananchi TZ: Vodacom M-Pesa, Tigo Pesa, Airtel Money, bank transfer, retail agents

4. **Dual-write phase** (during cutover transition, if the redeployed codebase has any pre-existing wallet capability): both the pre-existing wallet handling and the new BIL-05 receive the same gateway webhooks. Topups are credited to both; balances are reconciled daily. Discrepancies (rare) are operator-resolved. Duration: 2-4 weeks per operator. Whether this phase is needed at all depends on the cartography phase findings.

5. **Cutover**: pre-existing wallet handling (if any) is taken offline. BIL-05 becomes the system of record. BIL-03 starts calling BIL-05 for cycle debits; BIL-04 starts consuming `WalletToppedUp` events.

6. Per-operator activation order: start with Wananchi KE (largest segment, most mature gateway integrations), then Wananchi UG (smaller segment, simpler gateways), then Wananchi TZ.

Mapping to TZ baseline (section B) — what V3 adds vs what the redeployed codebase already supports — will be filled in after codebase review.

13. Refund-to-wallet path from BIL-02 (cleaner than refund-to-external-channel for many scenarios).

V3.2 backlog (deferred)

Feature	When it would matter	V3.2 path
Yas Senegal PREPAID wallet	If Yas SN expands offering beyond per-cycle PREPAYMENT	Add `XOF` to allowed currencies + Senegalese gateway adapters (Orange Money, Wave)
Multi-currency wallet	If a customer with cross-tenant Subscriptions wants a single wallet	Restructure to per-Customer wallet with currency sub-accounts
Wallet expiry rules	If regulator requires unused balance to expire after N months	Add `expires_at` per credit transaction + nightly expiry scanner
Mid-cycle pay-as-you-go purchases	If PREPAID extra-data-bundle or pay-per-view TV is added	Extend debit types + integrate with product-purchase workflow
Card-token aliases	If recurring card-based topups are added	Add `CARD_TOKEN` alias type + tokenization adapter
Bank-account aliases for direct-debit	If direct-debit cycle charging is offered	Add `BANK_ACCOUNT` alias + direct-debit workflow
Wallet-to-wallet transfer	If customer-to-customer transfers are supported (peer top-up)	New transfer workflow + dual-write atomic transaction
Wallet held in trust account	If regulator requires customer wallet balances to be held in segregated bank account	Add reconciliation with bank-side trust account daily
Auto-debit threshold	If customers want "auto-charge my card when wallet balance < X"	Card-on-file + threshold scanner
Promotional wallet credit campaigns	If marketing wants bulk credit drops for customer retention	Bulk-credit API + campaign tracking